UNIVERSIDAD DEL VALLE DE GUATEMALA

Redes



Proyecto 1

Uso de un protocolo

Erick Guerra - 23208

GUATEMALA, 8 de septiembre del 2026

1. Introduccion

Este proyecto consistió en construir un chatbot de consola que funciona como anfitrión del protocolo MCP, conectándose a distintos servidores para extender las capacidades de un modelo de lenguaje de Anthropic. El tema elegido fue un asistente para un centro de operaciones satelitales, capaz de responder preguntas sobre satélites en órbita, sus pasadas visibles desde una ubicación determinada y el riesgo de acercamiento con otros objetos catalogados.

El proyecto se desarrolló por fases, empezando por un chatbot funcional con memoria de conversación, siguiendo con la integración de servidores oficiales y propios, la incorporación de servidores creados por compañeros de curso, un servidor remoto desplegado en la nube y finalmente este informe.

2. Descripción general del sistema

El chatbot mantiene el contexto de la conversación entre turnos y registra cada interacción con los servidores MCP en la consola y en un archivo de registro. Al iniciar, se conecta a todos los servidores declarados en un archivo de configuración y expone sus herramientas al modelo de lenguaje, que decide cuándo usarlas según la pregunta del usuario.

Además de la interfaz de consola, que es la forma principal y obligatoria de usar el proyecto, se construyó una interfaz web opcional con el mismo motor de conversación, que muestra en tiempo real las llamadas a las herramientas y sus resultados, y permite adjuntar archivos locales, como un archivo de audio, para que las herramientas los analicen.

Un aspecto central del proyecto fue implementar el protocolo MCP de forma manual, sin depender del kit de desarrollo oficial de Anthropic para hablar JSON RPC. Esto se hizo tanto del lado del chatbot, que ahora arma y lee los mensajes JSON RPC directamente sobre entrada y salida estándar y sobre HTTP, como del lado del servidor satelital propio, que registra sus herramientas y responde a las peticiones sin usar el kit de desarrollo.

3. Especificación de los servidores MCP desarrollados

3.1 Servidor satelital local

Este servidor calcula información orbital real a partir de datos públicos de Celestrak y del algoritmo de propagación SGP4, el mismo modelo que se usa de forma operativa para el seguimiento de satélites. Se ejecuta de forma local, como un proceso que el chatbot lanza y con el que se comunica mediante mensajes JSON RPC por entrada y salida estándar. Ofrece tres herramientas.

Herramienta	**Proposito**	**Parametros principales**
Pasadas visibles	Calcula las próximas veces que un satélite es visible desde una ubicación dada.	Nombre o identificador del satélite, latitud y longitud del observador, elevación sobre el nivel del mar, días a calcular, elevación mínima sobre el horizonte.
Riesgo de conjunción	Evalúa si dos objetos catalogados corren riesgo de un acercamiento cercano en un rango de tiempo dado.	Nombre o identificador de ambos objetos, hora de inicio del rango, horas a analizar, resolución temporal del cálculo.
Parámetros orbitales	Traduce los datos orbitales crudos de un satélite a información legible, como inclinación, altitud y periodo.	Nombre o identificador del satélite.

Este servidor está publicado en un repositorio independiente y público de GitHub, con su propia documentación e instrucciones de instalación y uso.

3.2 Servidor satelital remoto

Para cumplir con el requisito de un servidor MCP que se ejecute realmente en la nube, se desplegó un segundo servidor satelital en Cloudflare Workers. Su función es sencilla, ya que el

objetivo de esta parte del proyecto es demostrar que el chatbot puede comunicarse con un servidor que no corre en la misma máquina, hablando el protocolo sobre una dirección web en lugar de un proceso local.

Herramienta	**Proposito**	**Parametros**
Ubicacion de la Estación Espacial Internacional	Devuelve la posición actual de la estación, incluyendo latitud, longitud, altitud y velocidad, a partir de una fuente pública en tiempo real.	No requiere parámetros.

Este servidor habla el protocolo sobre HTTP en su variante de transporte continuo, en la cual las peticiones se envían por HTTP normal y las respuestas pueden llegar como un solo mensaje o como un flujo de eventos.

4. Otros servidores MCP utilizados

Además de los dos servidores propios, el chatbot se conecta a otros cinco servidores para cumplir con las distintas fases del proyecto.

Los servidores oficiales de Anthropic para manejo de archivos y control de versiones permiten que el chatbot cree carpetas, escriba archivos y haga commits de git a partir de instrucciones en lenguaje natural, dentro de una carpeta de trabajo aislada.

También se integraron tres servidores creados por compañeros del curso. El primero administra reservaciones de un hotel, incluyendo disponibilidad de habitaciones y un flujo de reservación en dos pasos, donde el chatbot exige que el usuario vea una vista previa antes de confirmar cualquier cambio real. El segundo administra el catálogo de una tienda de renta de películas, con búsqueda, disponibilidad y recomendaciones. El tercero analiza mezclas de audio, midiendo el volumen percibido según el estándar de la industria, la distribución de frecuencias y la presencia de saturación digital, y se integró como un extra además de los dos servidores de compañeros

que pedía el proyecto.

5. Conclusiones y comentarios

El proyecto permitió ver de forma práctica cómo funciona un protocolo de comunicación real entre un cliente y varios servidores independientes, cada uno con su propia lógica interna pero compartiendo el mismo formato de mensajes. Trabajar con distintos transportes, tanto entrada y salida estándar como HTTP, ayudó a entender que el protocolo en sí es independiente del medio por el cual viaja.

Durante el desarrollo surgieron varias dificultades. El servidor oficial de git nunca implementó la creación de un repositorio desde cero, así que el escenario de demostración se ajustó para partir de una carpeta ya inicializada. Uno de los servidores de compañeros construía rutas de archivo de forma que el símbolo numeral en el nombre de la carpeta del proyecto rompía la conexión a su base de datos. También fue necesario agregar en el chatbot, y no en el servidor, la lógica que impide confirmar una reservación sin que el usuario haya visto antes una vista previa en un turno anterior de la conversación, ya que un servidor no tiene forma de saber por sí solo en que momento termina un turno de usuario y empieza otro.